



Exercise: XOR Cipher App

Overview

In this exercise, students will practice working with the **curses** TUI, file loading, C/Rust binding, and benchmarking by building an application to apply and benchmark a simple XOR cipher in Python and Rust. Note that you will need to use the **WSL terminal** (not PyCharm) to run and test this application!

Requirements

The application will use the **curses** module to provide a text-based interface for users. It will load text, apply a cipher variant, and benchmark variants against one another. The driver and Python routines will be in **cipher.py** and the Rust cipher in **xorcipher.rs**. Curses windowing calls (not merely drawing) should be invoked with box-drawing characters (not ASCII). Output should go to standard output unless otherwise specified.

Driver Program

When run directly, the program should...

- 1) Not directly invoke the `init scr()` function.
- 2) Display welcome message and menu
- 3) Initialize default text and key values (Fig. 1)
- 4) Show current text and key values under menu
- 5) Show status message (left-aligned) on 25th line
- 6) Detect **single-character input** for menu
- 7) Accept upper and lowercase input variants
- 8) Use a 25x80 (row/column) layout (Fig. 1)
- 9) Have **no global variables** (except functions)
- 10) Run the application if invoked directly – and only run the application when so invoked

```

Welcome to the XOR-Cipher App!

[F] Read text from a local File
[I] Read text from user Input prompt
[R] Apply Rust cipher to this text
[P] Apply Python cipher to this text
[V] Verify cipher results match
[K] Change Key used for ciphers
[B] Run Benchmarks on text (100000x)
[Q] Quit the Application

TEXT [This is a haiku; it is not too long I think; but you may disagree]
KEY  [But there's one sound that no one knows... What does the Fox say?]

Status: Application started successfully.

```

Fig. 1: Application upon initial execution (exactly as shown); Menu box: 10x40 (r,c)

Reading Strings from the User

Whenever a string is read from the user, the same dialog structure should be used (Figure 2).

The dialog should be formatted as follows:

- 1) Start on 18th row, 2nd column and be 6 x 78
- 2) Display prompt, centered, on 2nd row
- 3) Display box, centered, under prompt, size 3x68
- 4) Allow entry of 65 characters on one line only
- 5) Trim resulting whitespace at beginning / end
- 6) Terminate on entry of the ENTER / linefeed
- 7) All text should be manipulated as **CP437**
- 8) The prompt must match the specification
- 9) The status should be updated upon completion
- 10) If an empty string is entered, this should be considered a cancellation action by the user.

```

Welcome to the XOR-Cipher App!

[F] Read text from a local File
[I] Read text from user Input prompt
[R] Apply Rust cipher to this text
[P] Apply Python cipher to this text
[V] Verify cipher results match
[K] Change Key used for ciphers
[B] Run Benchmarks on text (100000x)
[Q] Quit the Application

TEXT [This is a haiku; it is not too long I think; but you may disagree]
KEY  [But there's one sound that no one knows... What does the Fox say?]

Enter file to load below, then press [ENTER]

Status: Application started successfully.

```

Fig. 2: Application displaying the dialog to load a file; Text Entry box: 3x68 (r,c)

NOTE: The curses textbox adds an extra space at the end of the input – make sure to test thoroughly!

Files should be read in text mode with assumed encoding of CP437, allowing for the assumption of 8-bit characters with a range of 0-255. The dialog prompt and status updates upon completion should be as follows:

If the filename is too long to display, the status line should be truncated to a length of 78 characters.

The text and key input dialogs for direct entry should be as follows:

The ciphers, in Python and Rust, will apply a variant of the XOR-Cipher on a *byte sequence* that can use a message and key of any non-zero length. The cipher is straightforward, and a Python variant is provided here:

While the cipher is simple, some characters cannot be displayed correctly without adjustment – particularly, the first 32 “control characters”. Symbols were developed to represent these characters graphically. To remove control characters and replace them with the symbolic counterparts, a translation can be used, as follows:

This translation should be used whenever displaying text in order to avoid problematic characters!



For verification, the current text and key should be run through the ciphers – *without updating the text* – and the results should be compared to verify that the cipher-text from each match one another.

Note that while your Rust library may work, testing with another library (one that fails) is fair game!

Cipher Benchmark

To benchmark the ciphers, each should be run 100,000 times using the **timeit** module, and the results should be displayed. The dialog should appear immediately, just before the benchmark begins, displaying a note that the benchmark is running (see below). Once the benchmark is complete, the dialog should be updated with the data. *This benchmark data should disappear from the screen as soon as another action is taken.*

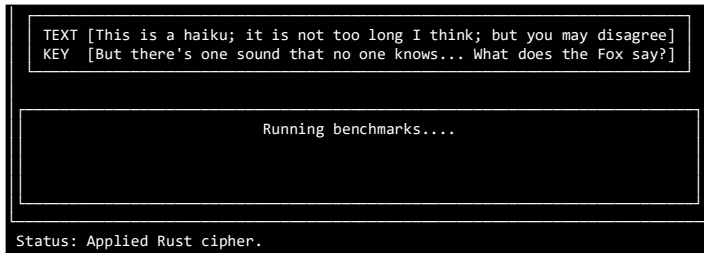


Fig. 5: Dialog when benchmarking is in progress; Response box: 6x78 (r,c)

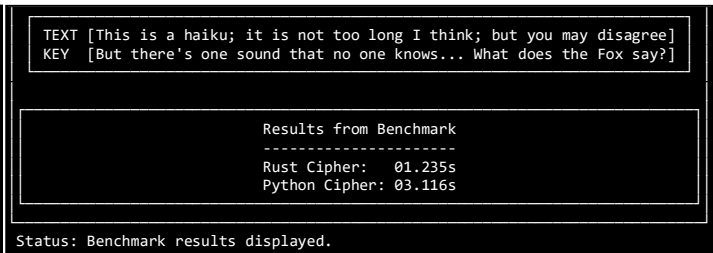


Fig. 6: Dialog when benchmarking has completed

Invalid Menu Selection

If an invalid character is pressed at the menu, the status should be updated to:
"Status: ERROR: Invalid menu selection!"

Termination

When the user elects to quit, the windowing system should shut down. On the ordinary text console, a goodbye message should be printed: **"Thanks for using the XOR-Cipher App; See you next time!"**

Structure

The following functions *must be present*. Python functions must be public and in the root of the module.

Python (cipher.py)

run_gui(background)

Takes background (typically **stdscr**) object as a parameter and runs the program. It should be runnable using **curses.wrapper()**. The function must have **no default parameter** and should not return anything.

cipher(message, key)

Executes the Python cipher described earlier. It should accept two **byte sequences** and return a ciphered **byte sequence**. It should not change the data stored in **message** or **key**.

load_cipher_lib(library_path)

Loads the cipher shared library at **library_path**, set its method parameters, and return the **library object**.

Rust (xorcipher.rs)

pub fn cipher(msg: *const i8, key: *const i8, buf: *mut i8, msg_len: usize, key_len: usize)

Executes the Rust-based cipher. The cipher-text should be written to **buf** by applying XOR to **key** and **msg**.

Submissions

NOTE: Your output must match the example output **exactly**. If it does not, *you will not receive full credit for your submission!* We will use the included **Makefile** to build and test your submission:



Please submit only and exactly these files:

Files: cipher.py, xorcipher.rs
Method: Submit on Canvas